

PUSH DOWN AUTOMATA:

normal FA + Stack = Push Down Automata

↓
static
indicator

↘
unbounded
stack

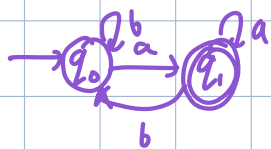
PDA

↳ is a tuple:

$(Q, \Sigma, \Gamma, \delta, q_0, \{f\}, z_0)$

* NDPDA → Non-deterministic Push Down Automata.

Consider a DFA, $L = \{w \mid w \text{ ends with } a\}$ $\Sigma = \{a, b\}$

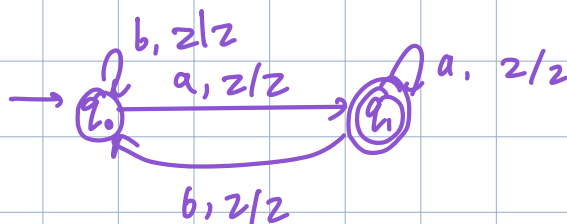


Now we have,



Here in the above case we do not need to worry about count, so no need of stack.

∴ NDPDA :

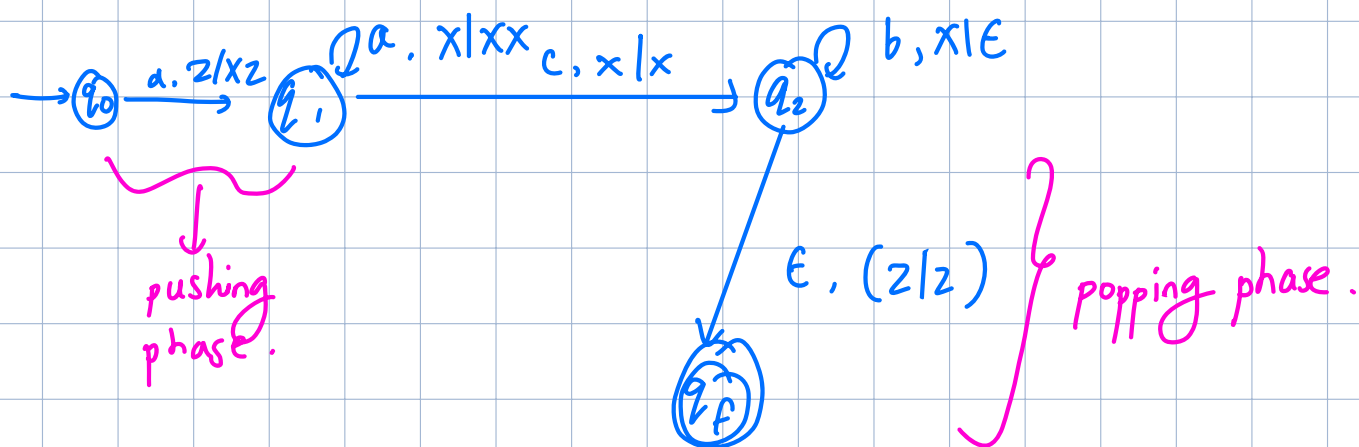


→ the following transitions are valid in a PDA,

$X \rightarrow \underbrace{1/X}_{\text{push}}$ $X \rightarrow \underbrace{\epsilon}_{\text{pop}}$ $X \rightarrow X$ no change.

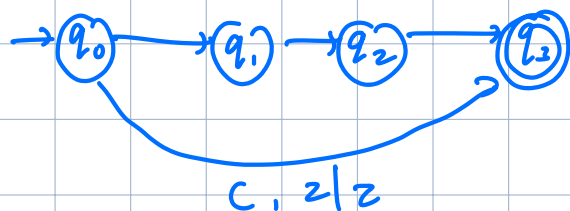
Q 1. Write a PDA for the language, $L = \{w \mid a^n c b^n, n \geq 1\}$, $w \in \{a, b, c\}^*$

Solⁿ $L = \{acb, acbb, aaacbbb, \dots\}$



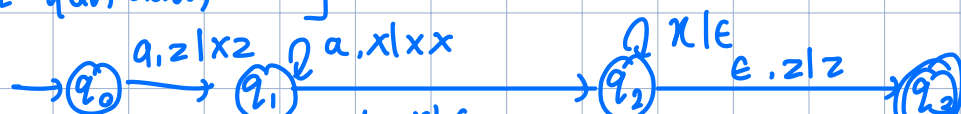
Q 2. Write PDA for $L = \{w \mid a^n c b^n, n \geq 0\}$, $w \in \{a, b, c\}^*$

Solⁿ $L = \{c, acb, aacbb, \dots\}$



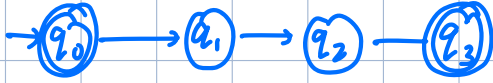
Q 3. $L = \{w \mid a^n b^n, n \geq 1\}$

Solⁿ $L = \{ab, aabb, \dots\}$



Q 4. $L = \{w \mid a^n b^n, n \geq 0\}$

Soln

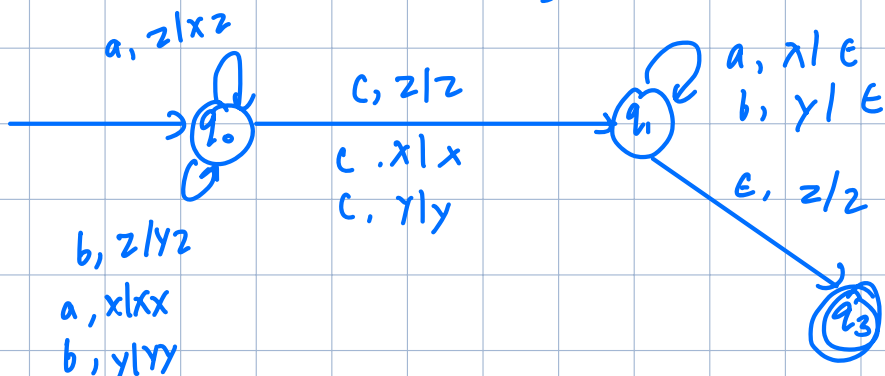


Q 5. $L = \{w \mid a^i b^j c^k, i+j=k, i, j, k \geq 1\}$

Soln $L = \{abcc, aabccc, abbccc, \dots\}$

Q 6. $L = \{w c w^R \mid w^R \text{ is reverse of } w\} \quad \Sigma = \{a, b\}$

Soln $L = \{c, aca, bcb, \dots\}$



a, y | xy

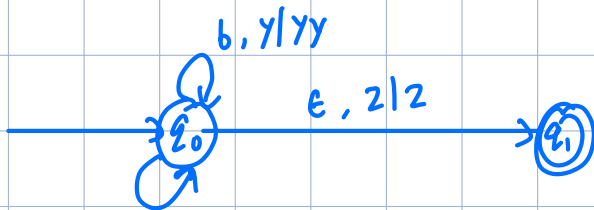
b, x | yx

Q 7. $L = \{w \mid n_a(w) = n_b(w)\}$

Solⁿ

$L = \{\epsilon, ab, ba, b^n a^n, a^n b^n, \dots\}$

$\Sigma = \{a, b\}$



a, y | ϵ

b, x | ϵ

a, z | xz

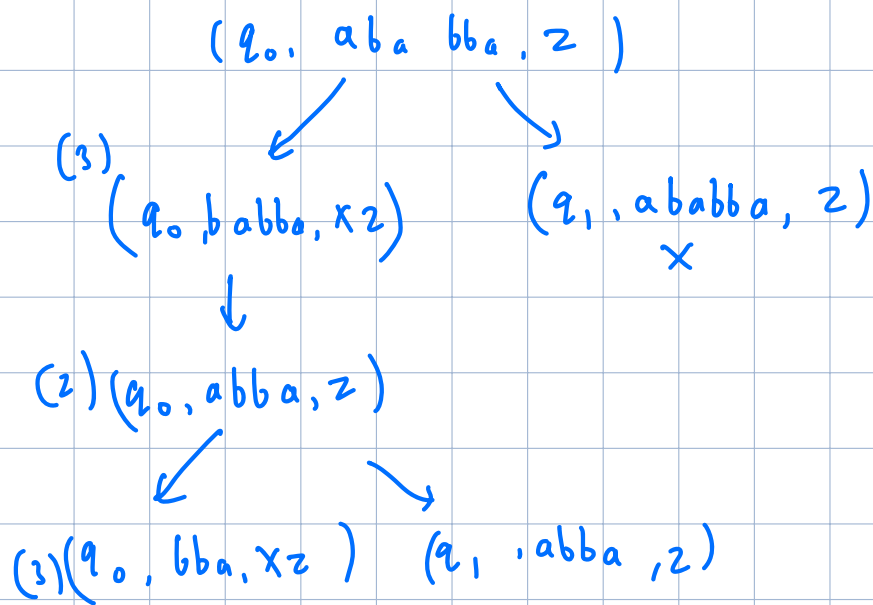
b, z | yz

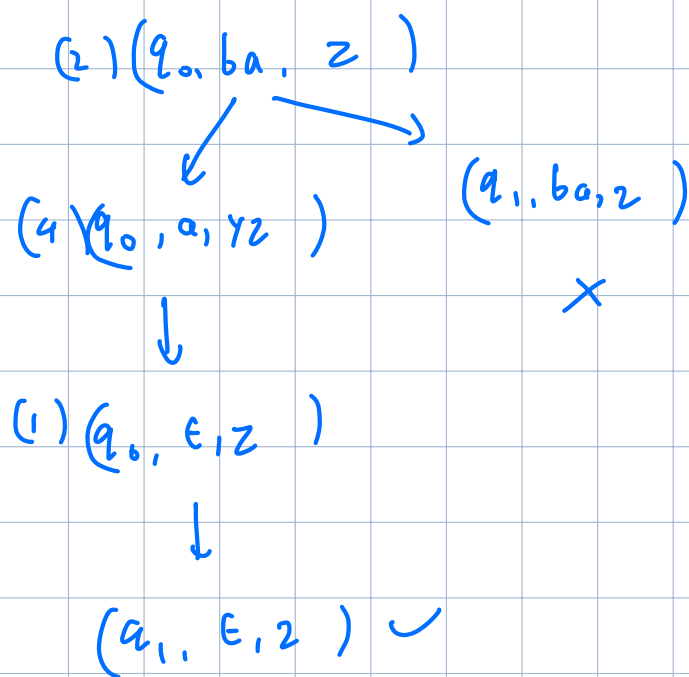
a, x | xx

In this example, there is no segregation of push & pop operation.

1D $\rightarrow$ Instantaneous Description.

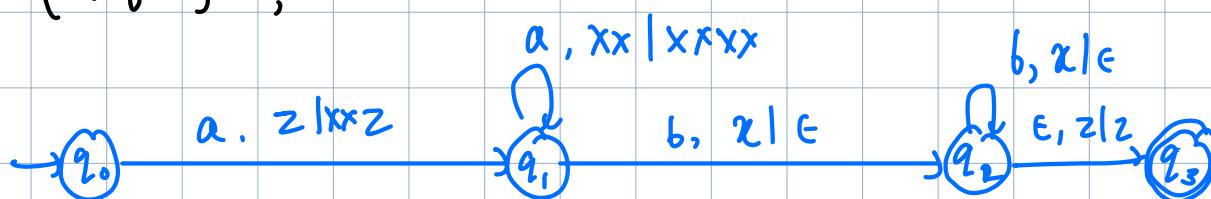
consider $ababba$





Q 8. $L = \{a^n b^{2^n}\}, n \geq 1$

Solⁿ



⇒ Definition: NPDA :

A non-deterministic push down acceptor is defined by the 7-tuple

$$M = (Q, \Sigma, q_0, F, \delta, \tau, z)$$

where,

Q is a finite set of internal states of the Control Unit

Σ is the input alphabet

τ is a finite set of symbols called the stack alphabet

$z \in \Gamma$ is the stack start symbol

$F \subseteq Q$ is the final states

$q_0 \in Q$ is the initial state of Control Unit

$\delta: Q \times (\Sigma \cup \{\epsilon\}) \times \Gamma \rightarrow \text{Finite subsets of } Q \times \Gamma^*$

The language accepted by the pushdown automata:

let, $M = (Q, \Sigma, \Gamma, \delta, q_0, F, z)$, M is an NPDA,

the language accepted by M is the set.

$$L(M) = \{w \mid w \in \Sigma^* : (q_0, w, z) \xrightarrow{*} (p, \epsilon, v), \\ p \in F, v \in \Gamma^*\}$$

$\Rightarrow$ Definition: DPDA:

A pushdown automaton $M = (Q, \Sigma, \Gamma, \delta, q_0, z, F)$ is said to be deterministic, if its subject to the restrictions that for every, $q \in Q$, $a \in \Sigma \cup \{\epsilon\}$, $b \in \Gamma$

1. $\delta(q, a, b)$ contains atmost 1 element
2. $\delta(q, \epsilon, b)$ is not empty then $\delta(q, c, b)$ must be empty for every $c \in \Sigma$

Note: • for any given input symbol & any stack top atmost one move can be made

• When a ϵ move is possible for some configuration then no/p consuming alternative is available.

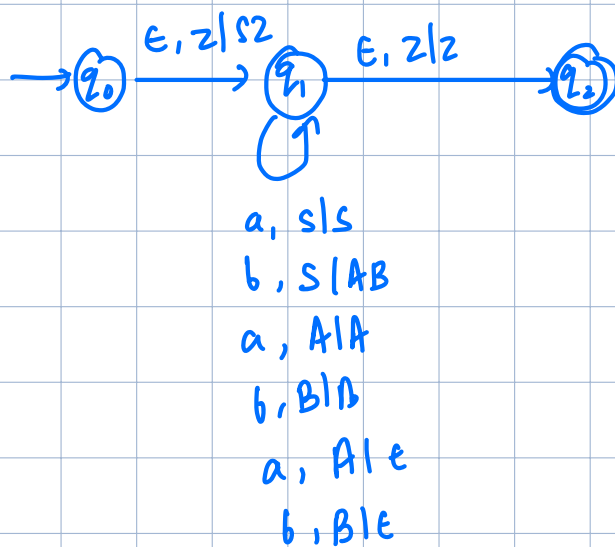
⇒ Construction of PDA for given CFG:

Q. 9 $S \rightarrow aS \mid bAB$

$A \rightarrow aA \mid a$

$B \rightarrow bB \mid b$

Soln } Convert CFG to CNF
already in CNF



⇒ Pumping Lemma for CFG:

Let L be an infinite CFL then there exists some tve integer m , such that any $w \in L$ with $|w| > m$. can be decomposed as $w = uvxyz$

with

(i) $|vxy| \leq m$

(ii) $|vy| \geq 1$

such that

To prove that a Language is Not Context Free using Pumping Lemma (for CFL) follow the steps given below: (We prove using CONTRADICTION)

- Assume that A is Context Free
- It has to have a Pumping Length (say P)
- All strings longer than P can be pumped $|S| \geq P$
- Now find a string ' S ' in A such that $|S| \geq P$
- Divide S into $uvxyz$
- Show that $u^i v^i x y^i z \in A$ for some i
- Then consider the ways that S can be divided into $uvxyz$
- Show that none of these can satisfy all the 3 pumping conditions at the same time
- S cannot be pumped == CONTRADICTION

$$u v^i x y^i z \in L \quad \forall i = 0, 1, 2, \dots$$

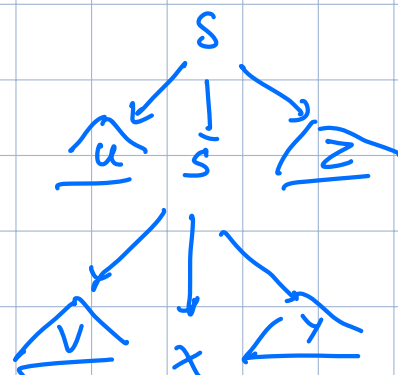
this is known as pumping lemma for CFL.

Q10. Consider $S \rightarrow ss|a|b$

Soln we can take $aaa bbb$

$$\begin{array}{ccccc} \frac{a}{u} & \frac{aa}{v} & \frac{b}{x} & \frac{b}{y} & \frac{b}{z} \end{array} \quad i=2$$

$$\simeq aaaaa bbb b \in L$$



Q11. Consider $a^n b^n c^n$

Soln take $aabbcc$

$$\frac{aa}{v} \quad \frac{b}{x} \quad \frac{b}{y} \quad \frac{cc}{z}$$

$$u v^2 x y^2 z = aaaa bbb cc \notin L$$

$\therefore a^n b^n c^n$ is not a CFL

⇒ Closure properties of Decision Algorithm for CFG:

- The family of CFL is closed under union, concatenation, star closure (Kleene star)

Note: (1) the family of CFL is not closed under intersection & complementation

(2) for every NPDA, there is no DPDA

Proof:

let L_1 and L_2 be 2 CFL's generated by CFG's

$$G_1 = (V_1, T_1, S_1, P_1)$$

$$G_2 = (V_2, T_2, S_2, P_2) \text{ respectively}$$

we can assume that without loss of generality that V_1 & V_2 are disjoint ($V_1 \cap V_2 = \emptyset$)

Now consider the lang, $L(G_3)$ generated by $G_1 \cup G_2$
 $\approx L(G_1) \cup L(G_2)$

$$\text{the } G_3 = (V_1 \cup V_2 \cup \{N, v\}, T_1 \cup T_2, N, P_1 \cup P_2 \cup \{x \rightarrow s, | A\})$$

$$q_1: s \rightarrow ssa|b$$

$$q_2: A \rightarrow aaAb$$

Similarly for concatenation,

$$q_4: x \rightarrow SA$$

$$G_4 = (V_1 \cup V_2 \cup \{N, v\}, T_1 \cup T_2, N, P_1 \cup P_2 \cup \{x \rightarrow SA\})$$

Similarly for Kleene star $G_5 = \begin{cases} S_1 \rightarrow S_1 S_1 \in \\ S \rightarrow ssa|b \end{cases} \cup G_1^*$

$$G_5 = (V_1, T_1, S_1, P_1 \cup \{S_1 \rightarrow \epsilon | S_1 S_1\})$$

For intersection:

Take L_1 & L_2 's intersection, then prove it's not CFL via pumping lemma.

Q.2 Convert to PDA.

$$S \rightarrow aSbb \mid aab$$

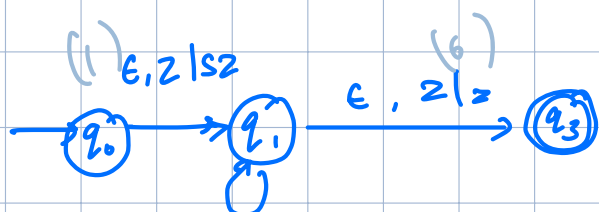
Solⁿ

$$S \rightarrow aSbb \mid aab$$

$$S \rightarrow aSx_1x_2b \mid a x_a x_b$$

$$x_1 \rightarrow b$$

$$x_a \rightarrow a$$

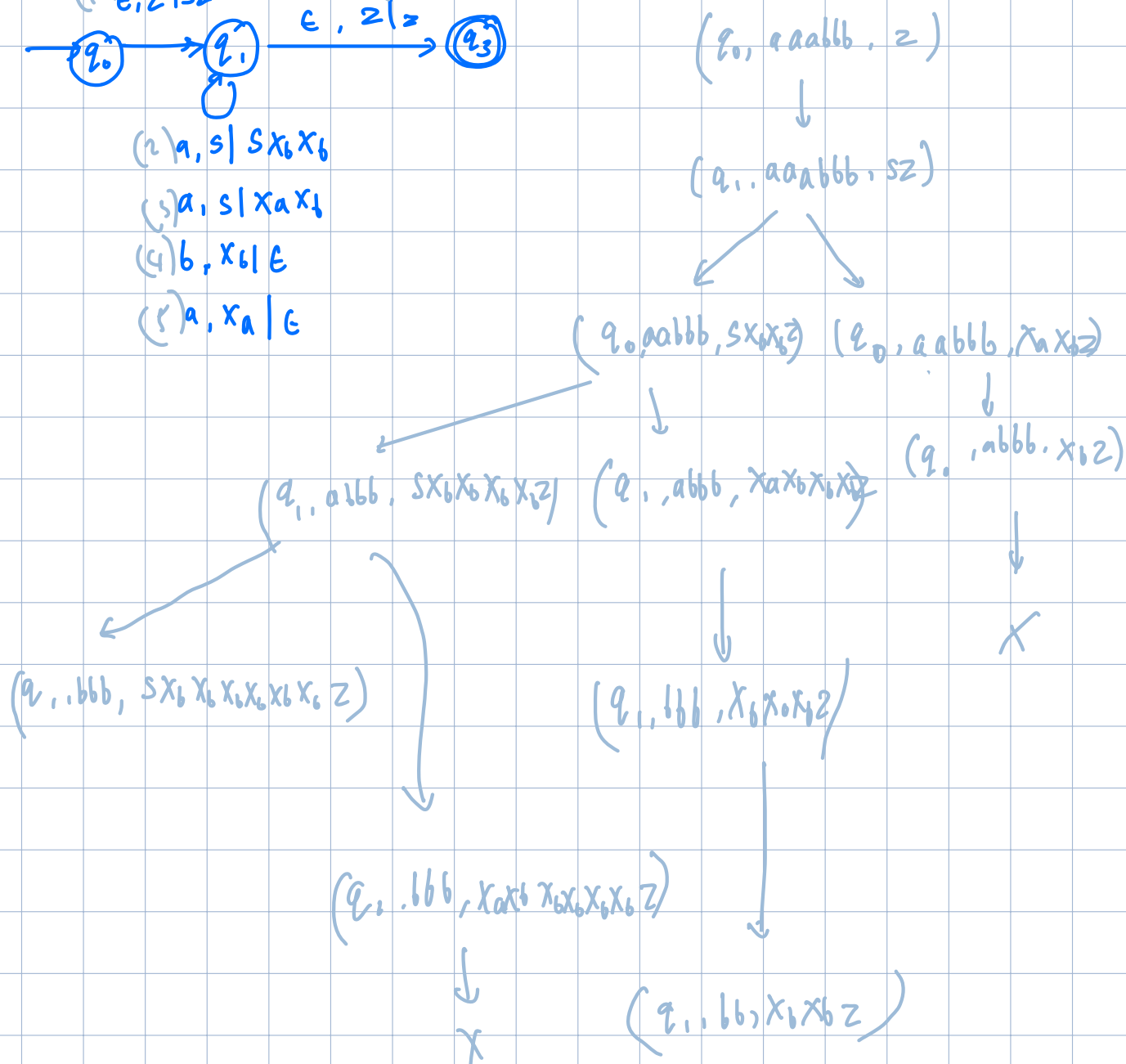


$$(2) a, s \mid Sx_1x_2b$$

$$(3) a, s \mid x_ax_b$$

$$(4) b, x_1 \mid \epsilon$$

$$(5) a, x_a \mid \epsilon$$

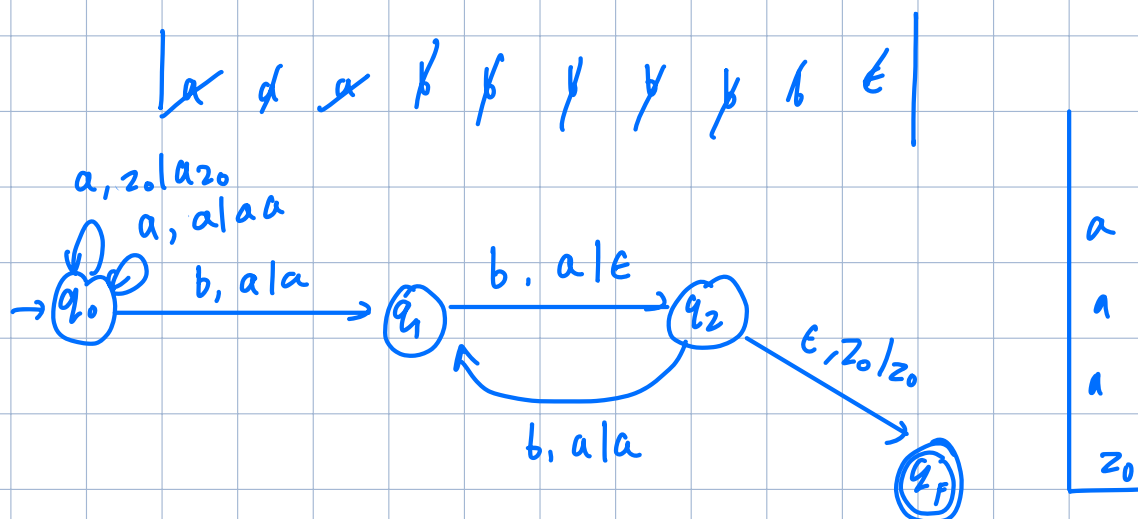


(q_1, b, x, z)
 $\downarrow (4)$
 (q_1, ϵ, z)
 $\downarrow (6)$
 (q_2, ϵ, z)

Q13. PDA for $L = \{a^n b^{2n}, n \geq 1\}$

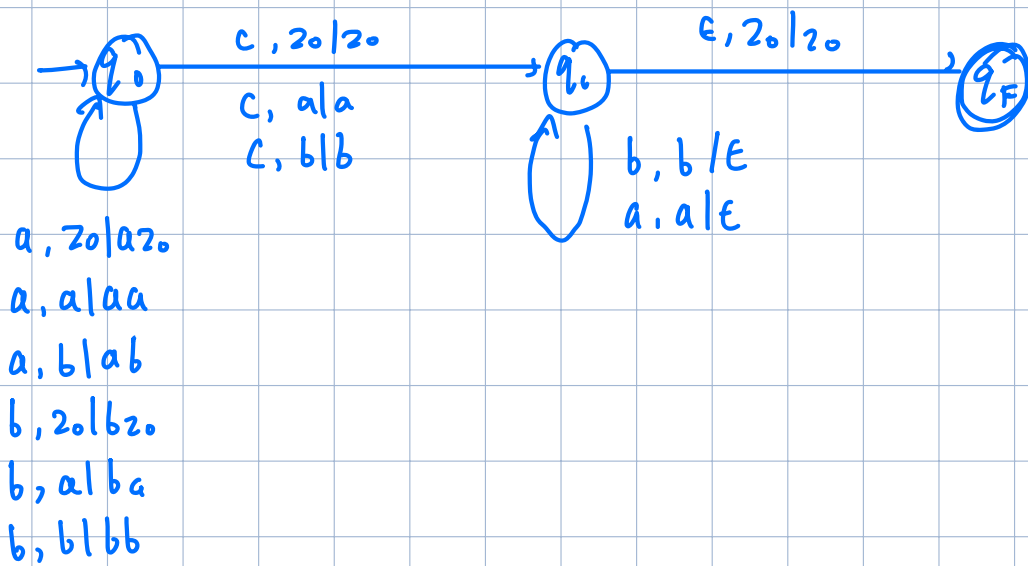
Solⁿ

let us take,



Q14. PDA for $L = \{w c w^R \mid w \in \{a, b\}^+\}$

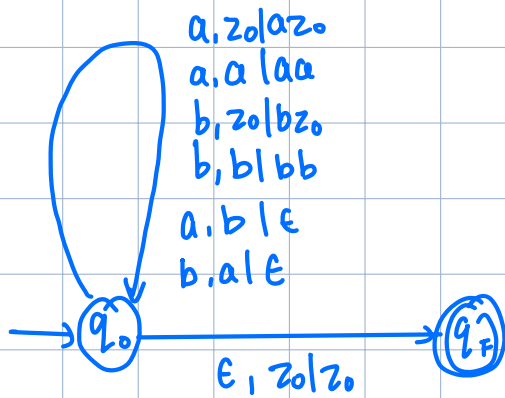
Solⁿ sample string: a b b c b b a ε



Note: $w c w$ is not a CFL

Q15. Construct PDA that accepts $L = \{ |w|_a = |w|_b \mid w \in (a,b)^* \}$?

Solⁿ



$| a \ b \ b \ a \ b \ a \ \epsilon |$

Stack contents (from top to bottom):
 $\cdot$
 b
 b
 a
 z_0

Q16. Construct PDA that accepts a language $L = \{ww^r \mid w \in (a,b)^*\}$?

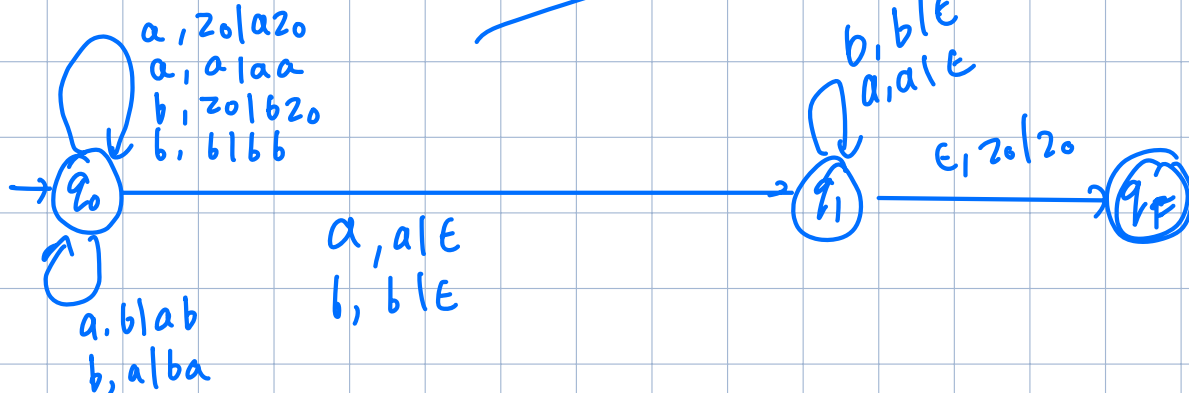
Soln

let $w = abb$

| a b b b b a ϵ |

NDPDA

b
b
a
z_0



ASSIGNMENT

① In each case below, say what language (a subset of $\{a,b\}^*$) is generated by the CFG with the indicated productions.

(a) $S \rightarrow aS \mid bS \mid \epsilon$

$L = \{a, aaa, aab, b, bb, \dots\}$

all strings over a and b , including the empty string.

(b) $S \rightarrow SS \mid bS \mid a$

one or more blocks each of the form $(b^n a)$. No empty string

$$(c) \quad S \rightarrow SaS \mid b$$

$$L = \{w \in \{a,b\}^* \mid n_b(w) = n_a(w) + 1, \text{ and for every prefix } v, n_b(v) \geq n_a(v)\}$$

$$(d) \quad S \rightarrow SaS \mid b \mid \epsilon$$

$$L = \{w \in \{a,b\}^*, n_b(w) - n_a(w) = 1 - t, t \geq 0\}$$

$$(e) \quad \begin{aligned} S &\rightarrow TT \\ T &\rightarrow aT \mid Ta \mid b \end{aligned}$$

All strings containing exactly 2 b's

$$(f) \quad \begin{aligned} S &\rightarrow aSa \mid bSb \mid aAb \mid bAa \\ A &\rightarrow aAa \mid bAb \mid a \mid b \mid \epsilon \end{aligned}$$

Strings built from palindromic middle pieces with symmetric outer wrapping.

$$(g) \quad \begin{aligned} S &\rightarrow aT \mid bT \mid \epsilon \\ T &\rightarrow aS \mid bS \end{aligned}$$

all even length strings over $\{a,b\}$, including the empty string.

$$\begin{aligned} (4) \quad S &\rightarrow aT \mid bT \\ T &\rightarrow aS \mid bS \mid \epsilon \end{aligned}$$

All non-empty odd-length strings.

② In each case below, find a CFG generating the given language:

(a) The set of odd-length strings in $\{a,b\}^*$ with middle symbol a .

Soln

$L = \{a, bab, baa, aaa, aab,$

$$S \rightarrow aSa \mid aSb \mid bSa \mid bSb \mid a$$

(b) Even-length strings in $\{a,b\}^*$ whose 2 middle symbols are equal.

$$S \rightarrow aSa \mid aSb \mid bSa \mid bSb \mid aa \mid bb$$

(c) Odd-length strings in $\{a,b\}^*$ whose first, middle, and last symbols are all same.

$L = \{aaa, bbb, ababa, babab, aaaaa, bbbbb, \dots\}$

3. Consider the productions in a CFG. For each, show that for every string $x \in L(G)$, $n_a(x) = n_b(x)$, then find a string $x \in (a,b)^*$ with $n_a(x) \neq n_b(x)$ that is not in $L(G)$

(a) $S \rightarrow SabS \mid SbaS \mid \epsilon$

(b) $S \rightarrow aSb \mid bSa \mid abS \mid baS \mid Sab \mid Sba \mid \epsilon$

Soln

(a) $S \rightarrow SabS \mid SbaS \mid \epsilon$

the base case for ϵ has $n_a(w) = 0$, $n_b(w) = 0$ so $n_a(w) = n_b(w)$

Each time we apply either $S \rightarrow SabS$ or $S \rightarrow SbaS$ you insert exactly one a and one b .

so. $n_a = n_b$ still satisfy

$$L(G) = (ab+ba)^*$$

$$aabb \notin L(G)$$

(b) $S \rightarrow aSb \mid bSa \mid abS \mid baS \mid Sab \mid Sba \mid \epsilon$

base case ϵ satisfies the condition

the grammar produces exactly 1 a and 1 b

so, $n_a = n_b$.

$$\text{no } x \notin L(G)$$

④ Consider the CFG

$$S \rightarrow aSbScS \mid aScSbS \mid bSaScS \mid bScSas \mid cSaSbS \mid cSbSa \mid \epsilon$$

Does this generate the language $\{x \in (a,b,c)^* \mid n_a(x) = n_b(x) = n_c(x)\}$?
Prove your answer.

We show by induction on the length $|w|$ that any nonempty w with $n_a(w) = n_b(w) = n_c(w)$ is derivable from S .

Base. $|w| = 0$: $w = \epsilon$ and $S \rightarrow \epsilon$ produces it.

Inductive step. Assume claim for all shorter strings. Let w be nonempty with $n_a(w) = n_b(w) = n_c(w)$. Consider the first symbol of w . By symmetry it suffices to treat the case the first symbol is a (the other cases b or c are identical up to permuting the roles of the letters). So write

$$w = a u.$$

Because the global totals of a and b are equal but we start with one more a than b in the prefix of length 1, the difference $\#a - \#b$ starts at $+1$ and finally becomes 0. Therefore there exists a *smallest* position p (with $p \geq 2$) such that the prefix $w[1..p]$ satisfies

$$\#_a(w[1..p]) = \#_b(w[1..p]).$$

That last letter of this prefix must be b (because the difference goes from $+1$ down to 0 and cannot be 0 earlier), so we can write the prefix as $a x b$. Thus

$$w = a x b y,$$

for some (possibly empty) x, y . Now consider the suffix y . In the whole string the counts of a and c are equal, and in the prefix $a x b$ the numbers of a and c may differ by some integer. Look at the running difference $\#a - \#c$ as we scan from the start of the suffix y . Initially it has some integer value, and finally (after the whole string) it must be 0. Thus there exists a (smallest) prefix of y where the running difference $\#a - \#c$ becomes zero; the last letter of that prefix must be c . So we can split

$$y = y_1 c z,$$

so

$$w = a x b y_1 c z.$$

Now set

$$u_1 := x, \quad u_2 := y_1, \quad u_3 := z.$$

By construction each block u_1, u_2, u_3 has strictly smaller total length than w . One checks the counts inside them satisfy

$$n_a(u_1) + n_a(u_2) + n_a(u_3) = n_a(w) - 1,$$

and similarly for b, c . Because we removed one a , one b and one c (the three outer letters), the three substrings u_1, u_2, u_3 together have equal totals of a, b, c . Moreover, by the way we chose the cut points (smallest prefixes where differences became zero), each substring u_i itself has equal numbers of a, b, c . (This is the standard "first-return" argument: the earliest time a particular pair of counts becomes equal ensures the substring between the corresponding boundary points is balanced for all three counts.) Thus each u_i is a strictly shorter string with equal numbers of a, b, c .

By the induction hypothesis each u_i is derivable from S . Therefore we can build w by the production

$$S \Rightarrow a S b S c S$$

(using the particular order $aSbScS$; if in the original scanning the balancing of b and c arose in the reverse order, we use the production $aScSbS$ instead). Concretely: derive u_1 from the first S , u_2 from the second S , and u_3 from the third S . That completes the inductive construction.

Because the argument used only the earliest prefix where a chosen pair of counts becomes equal, it covers all six cases of starting letter and orders of balancing; hence every nonempty w with equal counts can be produced by one of the six productions. This completes the induction.

⑤ Show that the language of all nonpalindromes over $\{a, b\}$ cannot be generated by any CFG in which

$$S \rightarrow aSa \mid bSb$$

Step 1: Analyze the structure of generated strings

Any derivation in G must start with S . The variable S can be replaced by one of the three forms of productions:

- **Case 3:** If the first step is $S \rightarrow w$, the generated string is w .
- **Case 1 or 2:** If the derivation uses productions (1) and (2) k times before using production (3), the derivation looks like:

$$S \Rightarrow x_1 S x_1 \Rightarrow x_1 x_2 S x_2 x_1 \Rightarrow \dots \Rightarrow x_1 x_2 \dots x_k S x_k \dots x_2 x_1$$

where each x_i is either a or b . Finally, S is replaced by a terminal string w :

$$x_1 x_2 \dots x_k S x_k \dots x_2 x_1 \Rightarrow x_1 x_2 \dots x_k w x_k \dots x_2 x_1$$

Let $x = x_1 x_2 \dots x_k$. The resulting string z is of the form:

$$z = xwx^R$$

Step 2: Show that all generated strings are palindromes

Let z be any string generated by G . The reverse of z , denoted z^R , is:

$$z^R = (xwx^R)^R$$

Using the property $(uv)^R = v^R u^R$:

$$z^R = (x^R)^R w^R x^R$$

Using the property $(u^R)^R = u$:

$$z^R = xw^R x^R$$

Now, consider the two sub-cases for the terminal production $S \rightarrow w$:

Case A: $S \rightarrow \epsilon$ (The terminal production is the empty string)

If $w = \epsilon$, then $z = x\epsilon x^R = xx^R$. Since $z^R = x\epsilon^R x^R = x\epsilon x^R = xx^R = z$, z is a palindrome.

Case B: $S \rightarrow w$, where w is a non-empty terminal string

Since w is the result of a terminal production (a base case in the recursion), for the overall string z to be a palindrome, we must have $z = z^R$.

$$xwx^R = xw^R x^R$$

This equality implies that $w = w^R$. **The base string w must be a palindrome.**

- If w is a single terminal, $w \in \{a, b\}$, then $w = w^R$.
- If w is a longer string, e.g., $w = aba$, then $w = w^R$.

⑦ What language over $\{a,b\}$ does the CFG with productions generate?

$S \rightarrow aas | bbs | sbb | abSab | absba | baSba | baSab | \epsilon$

$$L = \{w \in \{a,b\}^* \mid n_a(w) \cdot \frac{1}{2} = 0 \text{ and } n_b(w) \cdot \frac{1}{2} = 0\}$$

⑧ Find CFG assuming $i \geq 0, j \geq 0$

(a) $\{a^i b^j \mid i \leq j\}$

Set $L = \{e, b, ab, bb, abb, aabb, \dots\}$

$i=0, j=0 \Rightarrow e$

$j=1, i=0, 1, b, ab$

$j=2, i=0, 1, 2, bb, abb, aabb$

$S \rightarrow AB$

$A \rightarrow aAb \mid \epsilon$

$B \rightarrow bB \mid \epsilon$

$S \rightarrow \epsilon \mid aSB$

$B \rightarrow bB \mid b$

(b) $\{a^i b^j \mid i < j\}$

$S \rightarrow AB$

$A \rightarrow aAb \mid \epsilon$

$B \rightarrow bB \mid b$

$$(c) \{a^i b^j \mid j = 2i\}$$

$$a^i b^{2i} = a^i (b^2)^i \\ = a^i (bb)^i$$

$$S \rightarrow aSbb \mid \epsilon$$

$$(d) \{a^i b^j \mid i \leq j \leq 2i\}$$

ab or abb can be added at every expansion.

$$S \rightarrow aSb \mid aSbb \mid \epsilon$$

$$(e) \{a^i b^j \mid j \leq 2i\}$$

$$S \rightarrow \epsilon \mid aS \mid asb \mid asbb$$

$$(f) \{a^i b^j \mid j < 2i\}$$

$$S \rightarrow \epsilon \mid as \mid asb$$

9. Find a context-free grammar generating the language $\{a^i b^j c^k \mid i = j + k\}$

$$i = j + k$$

Soln

$$i = j + k$$

$$a^{j+k} b^j c^k$$

$$(a^k (a^j b^j) c^k)$$

$$S \rightarrow a S c \mid \epsilon$$

$$S_1 \rightarrow a S_1 b \mid \epsilon$$

(10) Let L be the language generated by the CFG with productions

$$S \rightarrow a S b \mid a b \mid S S$$

Show that no string in L begins with $ab b$

We show by induction on the structure of a derivation of S that every string derived from S satisfies (*).

Base case. If $S \Rightarrow ab$ then the prefixes are a and ab . For a we have $n_a = 1, n_b = 0$; for ab we have $n_a = n_b = 1$. So (*) holds.

Inductive steps.

1. Case $S \Rightarrow a S b$.

Let x be any string derived from the inner S . By the induction hypothesis x satisfies (*): every prefix u of x has $n_a(u) \geq n_b(u)$. Consider the string $w = a x b$.

- Any prefix that is just a has $n_a = 1, n_b = 0$, so OK.
- Any prefix that lies entirely inside $a x$ is a followed by a prefix u of x ; its counts are $n_a(\text{prefix}) = 1 + n_a(u)$ and $n_b(\text{prefix}) = n_b(u)$. By induction $1 + n_a(u) \geq 1 + n_b(u) > n_b(u)$, so the inequality holds.
- Any prefix that reaches into the final b has the form $a x v$ where v is a prefix of b (either empty or the full b). If it includes the final b , counts are $n_a = 1 + n_a(x)$ and $n_b = n_b(x) + 1$. But since x itself is balanced by construction of this grammar (every derivable string from S has equal numbers of a and b — see remark below) we have $n_a(x) \geq n_b(x)$, so $1 + n_a(x) \geq 1 + n_b(x) = n_b(x) + 1$. Thus every such prefix also satisfies (*).

So axb satisfies (*).

2. Case $S \Rightarrow S_1 S_2$.

Let w_1 and w_2 be strings derived from S_1 and S_2 . By the induction hypothesis both w_1 and w_2 satisfy (*). Also every string derived from S has equal numbers of a and b (this is immediate because both base ab and the wrap $a S b$ add one a and one b , and concatenation preserves equality), so in particular $n_a(w_1) = n_b(w_1)$.

Now consider any prefix v of $w = w_1 w_2$.

- If v is a prefix of w_1 , then v satisfies (*) because w_1 does.
- If v extends into w_2 , write $v = w_1 v_2$ where v_2 is a prefix of w_2 . Then

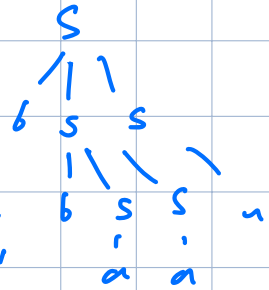
$$n_a(v) = n_a(w_1) + n_a(v_2), \quad n_b(v) = n_b(w_1) + n_b(v_2).$$

Since $n_a(w_1) = n_b(w_1)$ and v_2 satisfies $n_a(v_2) \geq n_b(v_2)$ (by induction on w_2), we get $n_a(v) \geq n_b(v)$. Thus all prefixes of w satisfy (*).

This completes the induction; hence every string in L satisfies (*).

⑪ What is the language produced by the CFG with productions

(a) $S \rightarrow a|as|bSS|ssb|SbS$



$$L = \{a, aa, aaa, baa, baaa, aaaaab, aaaaaba, \dots\}$$

containing $n_a > n_b$

(b) $S \rightarrow aSbS | aSbS | \epsilon$

$$\{w \in \{a,b\}^* \mid n_a(w) = n_b(w)\}$$

(c) $S \rightarrow aSaSbS | aSbSaS | bSaSaS | \epsilon$

$$\{w \in \{a,b\}^* \mid n_a(w) = 2n_b(w)\}$$

(d) $S \rightarrow SS | bTT | TbT | TTb | \epsilon$

$$T \rightarrow aS | SaS | Sa | a$$

$$\{w \in \{a,b\}^* \mid n_a(w) \geq n_b(w)\}$$

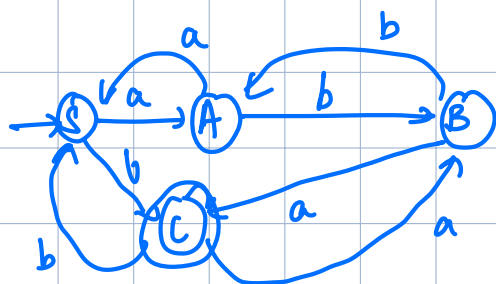
$$(e) S \rightarrow a|aS|bSS|SSb|SbS$$

$$\{w \in \{a,b\}^* \mid n_a(w) > n_b(w)\}$$

$$(f) \begin{aligned} S &\rightarrow SaT | \epsilon \\ T &\rightarrow TbS | \epsilon \end{aligned}$$

(12) In each part, draw an NFA accepting the language generated by the CFG having the given productions

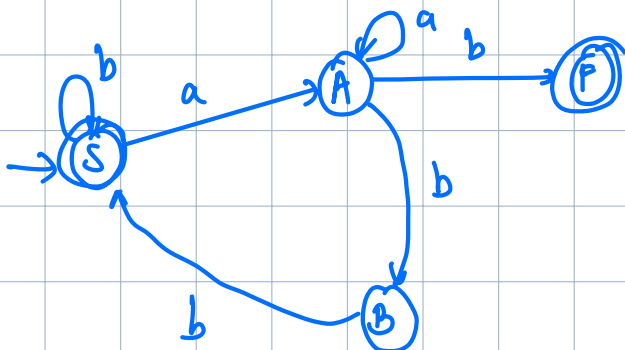
$$\begin{aligned} (0) \quad S &\rightarrow aA | bC \\ A &\rightarrow aS | bB \\ B &\rightarrow aC | bA \\ C &\rightarrow aB | bS | \epsilon \end{aligned}$$



(b) $S \rightarrow bS \mid aA \mid \epsilon$

$A \rightarrow aA \mid bB \mid b$

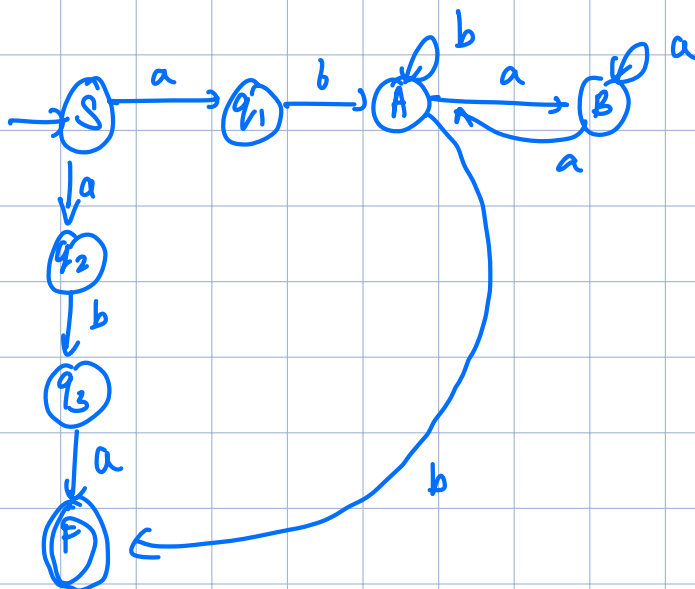
$B \rightarrow bS$



(c) $S \rightarrow abA \mid bB \mid aba$

$A \rightarrow b \mid aB \mid bA$

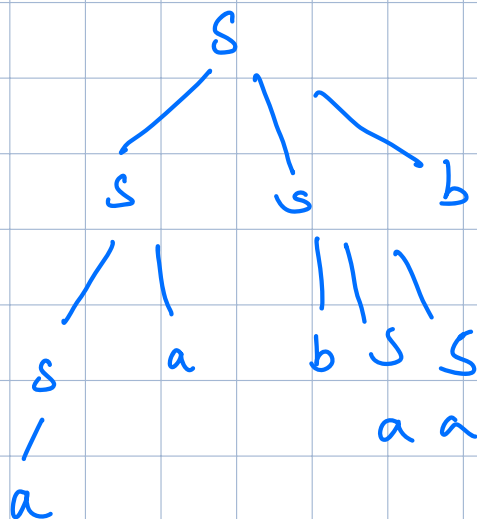
$B \rightarrow aB \mid aA$



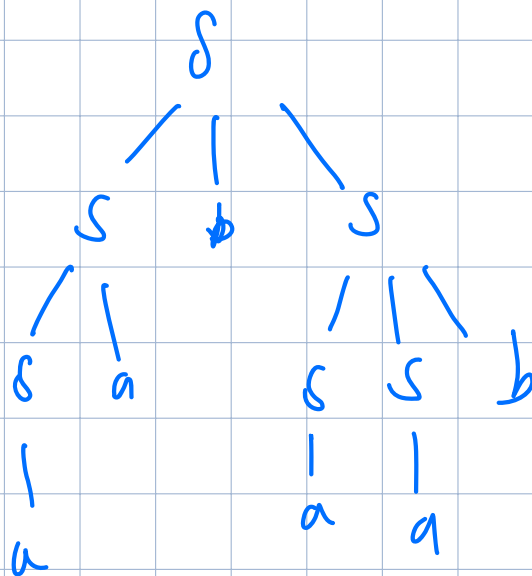
⑬ Show that the CFG with the following productions is ambiguous
 $S \rightarrow a \mid Sa \mid bS \mid SSb \mid SbS$

consider string $aabaab \in G$

①



②

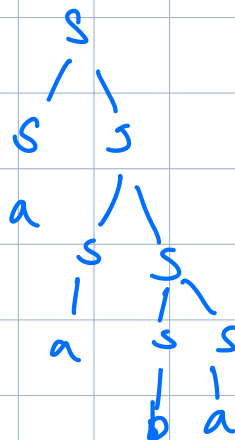
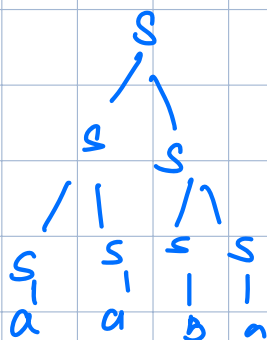


Since the grammar has 2 derivation trees,
it is ambiguous.

(14) In each case below, show that the grammar is ambiguous, and find an equivalent unambiguous grammar.

(a) $S \rightarrow SS \mid a \mid b$

$aabaa$



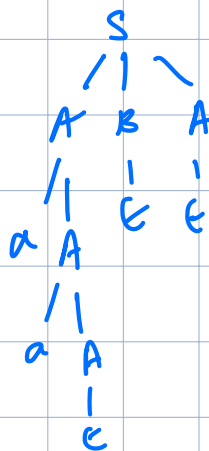
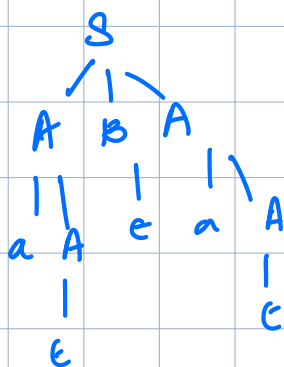
$S \rightarrow aS \mid bS \mid a \mid b$

(b) $S \rightarrow ABA$

$A \rightarrow aA \mid \epsilon$

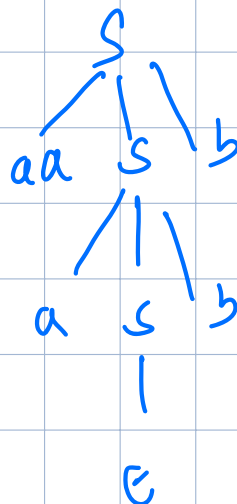
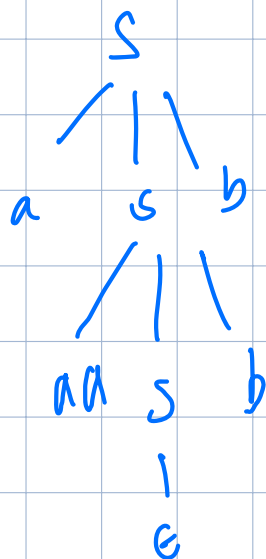
$B \rightarrow bB \mid \epsilon$

consider aa



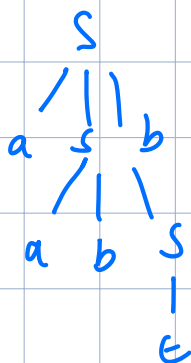
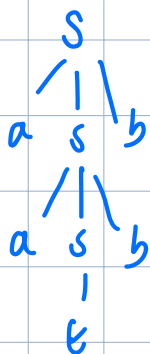
(c) $S \rightarrow aSb \mid aaSb \mid \epsilon$

consider $aaabbb$



(d) $S \rightarrow aSb \mid abS \mid \epsilon$

consider $aabb$



(5) In each case below, find CFG with no ϵ production.

(a) $S \rightarrow AB | \epsilon$
 $A \rightarrow aAsb | a$
 $B \rightarrow bS$

$$NV \rightarrow \{S\}$$

$$S \rightarrow AB | A | B$$
$$A \rightarrow aAsb | a$$
$$B \rightarrow bS$$

(b) $S \rightarrow AB | ABC$
 $A \rightarrow BA | BC | \epsilon | a$
 $B \rightarrow AC | CB | \epsilon | b$
 $C \rightarrow BC | AB | A | c$

$$NV \rightarrow \{A, B, C, S\}$$

$$S \rightarrow AB | ABC | A | B | AB | BC | AC$$
$$A \rightarrow BA | BC | a | A | B | C$$
$$B \rightarrow AC | CB | b | A | C | B$$
$$C \rightarrow BC | AB | A | c | B$$

(16) Remove ϵ and unit production.

(a) $S \rightarrow ABA$
 $A \rightarrow aA | \epsilon$
 $B \rightarrow bB | \epsilon$

Solⁿ (i) removing ϵ

$$NV = \{S, A, B\}$$
$$S \rightarrow ABA | B | AA$$
$$A \rightarrow aA | a$$
$$B \rightarrow bB | b$$

(ii) removing unit produ.

$$S \rightarrow ABA | bB | b | AA$$
$$A \rightarrow aA | a$$
$$B \rightarrow bB | b$$

(b) $S \rightarrow aSa | bSb | \epsilon$
 $A \rightarrow aBb | bBa$
 $B \rightarrow aB | bB | \epsilon$

$$NV = \{S, B\}$$

$$S \rightarrow aSa | bSb | aa | bb$$
$$A \rightarrow aBb | bBa$$

$$B \rightarrow aB \mid bB \mid a \mid b$$

No unit prod

$$(c) \quad S \rightarrow A \mid B \mid C$$

$$A \rightarrow aAa \mid B$$

$$B \rightarrow bB \mid bb$$

$$C \rightarrow aCaa \mid D$$

$$D \rightarrow baD \mid abD \mid aa$$

no ϵ produ.

$$S \rightarrow aAa \mid bB \mid bb \mid aCaa \mid baD \mid abD \mid aa$$

$$A \rightarrow aAa \mid bB \mid bb$$

$$B \rightarrow bB \mid bb$$

$$C \rightarrow aCaa \mid baD \mid abD \mid aa$$

$$D \rightarrow baD \mid abD \mid aa$$

(17) Remove useless variables.

$$(a) \quad S \rightarrow ABC \mid BaB$$

$$A \rightarrow aA \mid BaC \mid aaa$$

$$B \rightarrow bBb \mid a$$

$$C \rightarrow CA \mid AC$$

C is reachable but non-generating, so delete C.
if we delete C, A becomes non-reachable

$$S \rightarrow BaB$$

$$B \rightarrow bBb|a$$

$$(b) S \rightarrow AB|AC$$

$$A \rightarrow aAb|bAa|a$$

$$B \rightarrow bbA|aaB|AB$$

$$C \rightarrow abCa|aDb$$

$$D \rightarrow bD|aC$$

C and D are reachable but non-generating

$$S \rightarrow AB$$

$$A \rightarrow aAb|bAa|a$$

$$B \rightarrow bbA|aaB|AB$$

Q Remove ϵ transitions:

$$S \rightarrow aAB$$

$$A \rightarrow aAA|\epsilon$$

$$B \rightarrow bBB|\epsilon$$

Soln

$$S \rightarrow aAB|aA|aB|a$$

$$A \rightarrow aAA|aA|a$$

$$B \rightarrow bBB \mid bB \mid b$$

ASSIGNMENT-4:

- ① Find a CFG generating the given language with the set of odd-length strings in $\{a, b\}^*$ with middle symbol a .

Soln

$$L = \{a, aab, aaa, baa, bab, \dots\}$$

$$S \rightarrow aSa \mid bSb \mid aSb \mid bSa \mid a$$

- ② Write CFG for $L = a^n b^n, n$ is even

Soln

$$L = \{\epsilon, aabb, aaabbbb, \dots\}$$

$$S \rightarrow aaSbb \mid \epsilon$$

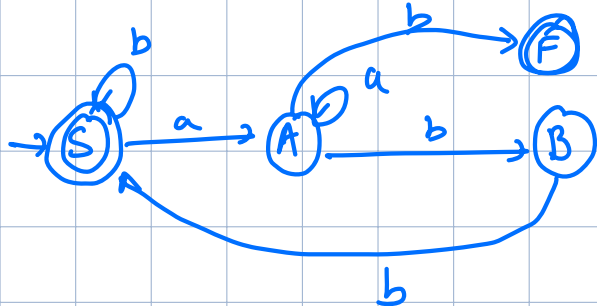
- ③ Draw FA for CFG:

$$S \rightarrow bS \mid aA \mid \epsilon$$

$$A \rightarrow aA \mid bB \mid b$$

$$B \rightarrow bS$$

Soln



④ Find a regular grammar generating the language :

$$S \rightarrow AAS \mid ab \mid aab$$

$$A \rightarrow ab \mid ba \mid \epsilon$$

Soln regex = $(ab+ba)^* (ab+aab)$

$$S \rightarrow abA' \mid aabA'$$

$$A' \rightarrow AA' \mid \epsilon$$

$$A \rightarrow ab \mid ba \mid \epsilon$$

$$S \rightarrow abA' \mid aabA'$$

$$A' \rightarrow abAA' \mid baAA' \mid \epsilon$$

$$A \rightarrow ab \mid ba \mid \epsilon$$

⑤ For the grammar $G = (\{S\}, \{a,b\}, S, P)$, with productions

$$S \rightarrow aSa$$

$$S \rightarrow bSb$$

$$S \rightarrow \lambda$$

Show a derivation tree for $w = aaabbaaa$



- ⑥ Define CNF. Give CNF for generating all properly nested parentheses.
A properly nested parentheses structures are the ones involving 2 kinds of parentheses, say $()$ and $[\]$: eg: $([\])$, $([\ [\]])$, $[\ (\)]$

$$S_0 \xrightarrow{J^n} S \rightarrow (s) \mid [s] \mid ss \mid \epsilon$$

- ⑦ Define Regular Grammar. Find a regular grammar generating:

(a) $S \rightarrow SSS | a | ab$

(b) $S \rightarrow AabB$

$$A \rightarrow aA \mid bA \mid \epsilon$$

$$B \rightarrow Bab \mid Bb \mid ab \mid b$$

$$(c) S \rightarrow \epsilon A(B$$

$$A \rightarrow AAA | Ab | bA | a$$

$$B \rightarrow bB | \epsilon$$

S.I'n

$$(a) S \rightarrow SSS | a | ab$$

↓

$$S \rightarrow aaS | aabS | ababS | abaS | a | ab$$

$$(b) S \rightarrow AabB$$

$$A \rightarrow aA | bA | \epsilon$$

$$B \rightarrow Bab | Bb | ab | b$$

$$(c) S \rightarrow \epsilon A(B$$

$$A \rightarrow AAA | Ab | bA | a$$

$$B \rightarrow bB | \epsilon$$

⑧ Find CFG for

$$(a) \{a^i b^j \mid j \leq 2i\}$$

Soln

$$i=0, j=0$$

$$i=1, j=0, 1, 2$$

$$i=2, j=0, 1, 2, 3, 4$$

$$L = \{\epsilon, a, ab, abb, aa, aab, aabb, aabb, aabbb, \dots\}$$

$$S \rightarrow \epsilon \mid asbb \mid aS \mid asb$$

$$(b) \{a^i b^j c^k \mid i \neq j+k\}$$

$$\begin{array}{l|l} i > j+k \quad j+k \geq 0 & i < j+k \\ S \rightarrow a \mid aaSbc \mid aaSb \mid aaSc & S \rightarrow bc \mid bcl \mid aSbcl \mid asb \mid asc \end{array}$$

$$S \rightarrow S_1 \mid S_2$$

$$S_1 \rightarrow a \mid aaS_1b \mid aaS_1c \mid aaS_1bc$$

$$S_2 \rightarrow bcl \mid bc \mid aS_2b \mid aS_2c \mid aS_2bc$$

$$(c) L = \{w : n_a(w) \neq n_b(w)\}$$

$$L = \{a, b, aab, baa, aba, \dots\}$$

$$S \rightarrow U | V$$

$$E \rightarrow \epsilon | EE | aEb | bEa$$

$$U \rightarrow a | aU | Va | UU | EU | UE$$

$$V \rightarrow b | bV | Vb | VV | EV | VE$$

$$(d) L = \{w : n_a(v) \geq n_b(v), \text{ where } v \text{ is any prefix of } w\}$$

Soln

$$S \rightarrow SS | aSb | aS | \epsilon$$

(e) The set of odd-length strings in $\{a, b\}^*$ whose first, middle, and last symbols are same.

Soln

$$S \rightarrow A | B$$

$$A \rightarrow a | aXa$$

$$B \rightarrow b | bYb$$

$$X \rightarrow a | aXa | aXb | bXa | bXb$$

$$Y \rightarrow b | aYa | aYb | bYa | bYb$$

⑨ Find a string $x \in \{a, b\}^*$ with $n_a(x) = n_b(x)$ that is not in $L(G)$

$$(a) S \rightarrow SabS \mid SbaS \mid \epsilon$$

Soln $L = \{ ab, ba, \dots \}$

ans aa bb

$$(b) S \rightarrow asb \mid bSa \mid baS \mid Sab \mid Sba \mid \epsilon$$

Soln no π

10) Find equivalent unambiguous grammar

$$(a) S \rightarrow a \mid sa \mid bSS \mid SSb \mid SbS$$

Soln

$$\begin{aligned} S &\rightarrow L \mid R \\ L &\rightarrow \epsilon \mid bL \\ R &\rightarrow \epsilon \mid aR \mid bR \end{aligned}$$

$$(b) S \rightarrow SS \mid a \mid b$$

Soln $S \rightarrow alb \mid as \mid bS$

$$(c) S \rightarrow ABA$$

$$A \rightarrow aA \mid \epsilon$$

$$B \rightarrow bB \mid \epsilon$$

Soln $a^* b^* a^*$

$S \rightarrow P$

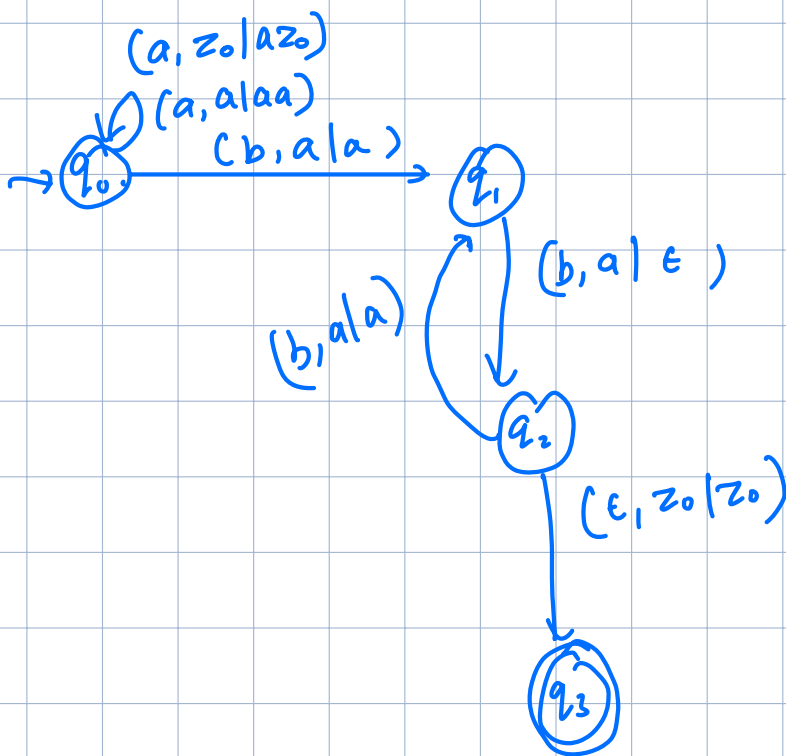
$P \rightarrow aP \mid Q$

$Q \rightarrow bQ \mid R$

$R \rightarrow aR \mid \epsilon$

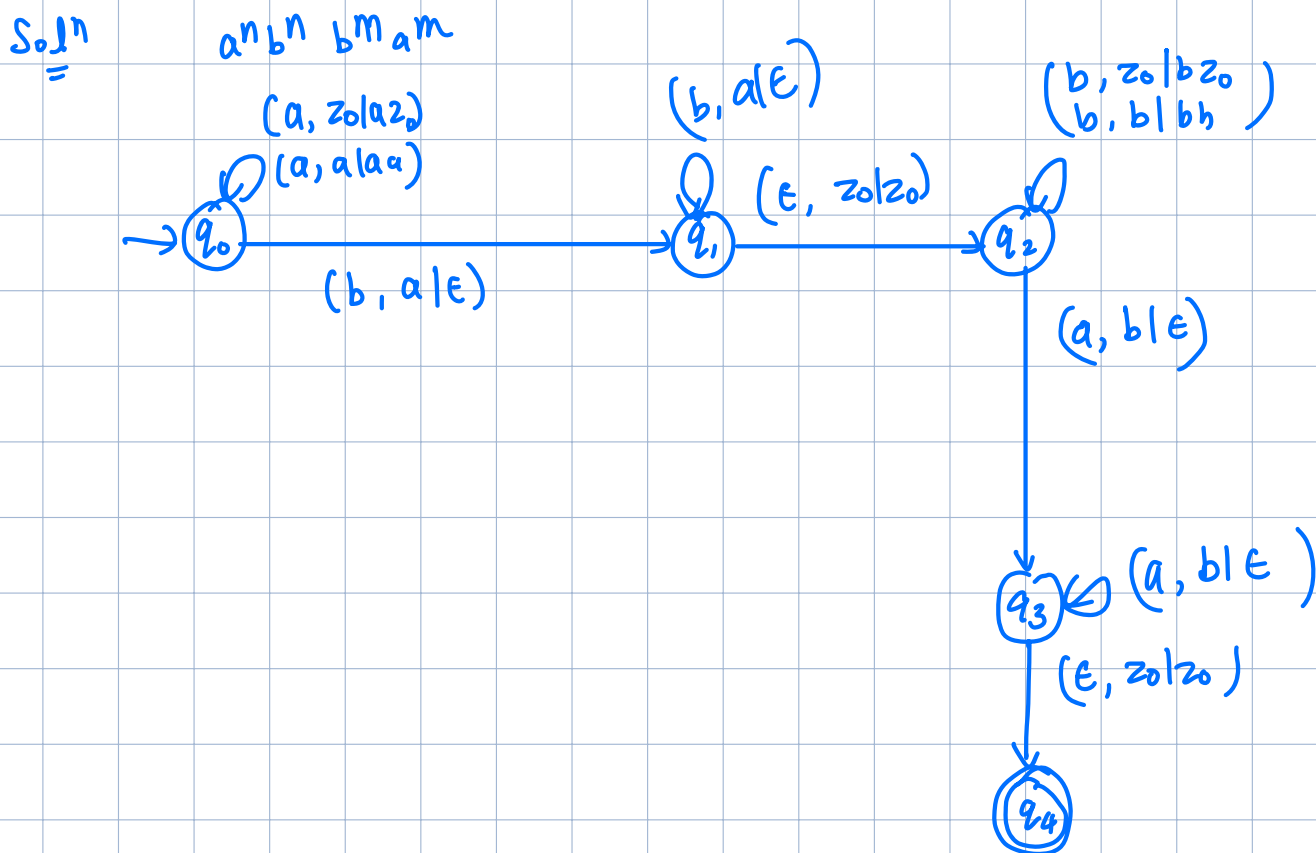
- ② Find PDA that accepts $L = \{a^n b^{2n} : n \geq 0\}$. Show the sequence of instantaneous descriptions for acceptance of $a^1 b^2$

Soln $| a \ a \ b \ b \ b \ b \ e |$



a
a
z ₀

12) Give a transition table for a deterministic PDA that accepts the language $L = \{a^n b^{n+m} a^m \mid n, m \geq 0\}$



$$(q_0, a, z_0) = (q_0, a z_0)$$

$$(q_0, a, a) = (q_0, a a)$$

$$(q_0, b, a) = (q_1, \epsilon)$$

$$(q_1, b, a) = (q_1, \epsilon)$$

$$(q_1, \epsilon, z_0) = (q_2, z_0)$$

$$(q_2, b, z_0) = (q_2, b z_0)$$

$$(q_2, b, b) = (q_2, b b)$$

$$(q_2, a, b) = (q_3, \epsilon)$$

$$(q_3, a, b) = (q_3, \epsilon)$$

$$(q_3, \epsilon, z_0) = (q_4, z_0)$$

⑮ Show using PL that the given language is not CFL.

$$(a) L = \{ ww \mid w \in \{0,1\}^* \}$$

Pumping lemma (CFL form).

If L is an infinite CFL then there is a pumping length $p \geq 1$ such that every string $s \in L$ with $|s| \geq p$ can be written

$$s = uvwxy$$

with $|vwx| \leq p$, $|vx| \geq 1$, and for every $i \geq 0$ the string $uv^iwx^iy \in L$.

We assume each language is context-free, get the pumping constant p , pick a particular long string $s \in L$ (with $|s| \geq p$), and show that no valid splitting $s = uvwxy$ can satisfy the pumping property for all i .
Contradiction $\Rightarrow$ language not CFL.

$$\text{consider } L = \{ 0^n 1^m 0^n \mid n, m \geq 0 \}$$

$$\text{let } s = 0^p 1^p 0^p$$

$$s = uvwxy \text{ with } |vwx| \leq p \text{ and } |vx| \geq 1$$

consider.

$$vwx = 1^p$$

$$v = 1^{k_1}$$

$$w = 1^{k_2}$$

$$x = 1^{p-k_1-k_2}$$

$$u = 0^p$$

$$y = 0^p 1^p$$

$$\text{let } i=2$$

$$0^p (1^{k_1})^2 1^{k_2} (1^{p-k_1-k_2})^2 0^p 1^p$$

$$= 0^p 1^{2k_1 k_2} 1^{2p-2k_1-2k_2} 0^p 1^p$$

$$= 0^p 1^{2p-k_2} 0^p 1^p \notin L$$

$\therefore ww$ is not CFL

$$(b) L = \{a^n b^{2n} a^n \mid n \geq 0\}$$

$$\text{let } s = a^p b^{2p} a^p$$

4. **Identify Cases based on** $|vxy| \leq p$: Since $|vxy| \leq p$, the segment vxy can only be contained within one of the three blocks (a^p, b^{2p}, a^p), or span the boundary between two adjacent blocks.

- **Case 1: vxy is contained entirely within a^p (the first block).**
 - v and y must consist only of a 's. Since $|vy| > 0$, pumping $i = 2$ (i.e., uv^2xy^2z) will increase the number of a 's in the first block, making the string $a^{p+|v|+|y|}b^{2p}a^p$.
 - This string is of the form $a^{n'}b^{2p}a^p$. For this to be in L , we must have $n' = p$ and $2p = 2n'$, which would require $p = p$ and $p = p$. But since $|v| + |y| > 0$, $n' = p + |v| + |y| > p$.
 - Since $n' \neq p$, the total count of a 's on the left does not equal the count of a 's on the right, so the string is not in the form $a^nb^{2m}a^n$. $s' \notin L$.
- **Case 2: vxy is contained entirely within b^{2p} (the middle block).**
 - v and y must consist only of b 's. Pumping $i = 2$ will increase the number of b 's: $a^pb^{2p+|v|+|y|}a^p$.
 - For this string to be in L , we need $2p + |v| + |y| = 2p'$, where p' is the number of a 's (which is p in this case).
 - This requires $2p + |v| + |y| = 2p$, which means $|v| + |y| = 0$. This contradicts the Pumping Lemma condition $|vy| > 0$. $s' \notin L$.
- **Case 3: vxy spans the boundary between a^p and b^{2p} (e.g., $v = a^*, y = b^*$ or vice-versa).**
 - Pumping $i = 2$ will interleave the a 's and b 's, or increase one group more than the other, resulting in a string not of the form $a^nb^{2m}a^n$. For example, if v has an a and y has a b , then v^2 and y^2 will break the required ordering. $s' \notin L$.

$$(c) L = \{a^n b^n c^n d^n \mid n \geq 0\}$$

$a^p b^p c^p d^p$. The length of s is $|s| = 4p$, which is clearly $\geq p$.

3. **Apply Pumping Lemma:** By the lemma, s can be divided into $s = uvxyz$ such that $|vxy| \leq p$ and $|vy| > 0$.
4. **Identify Cases based on** $|vxy| \leq p$: Since $|vxy| \leq p$, the segment vxy is too short to cross more than two adjacent blocks (e.g., it cannot span a^p and c^p).
 - **Case 1: vxy is contained entirely within two adjacent blocks (e.g., $a^p b^p$).**
 - v and y can only contain a 's and/or b 's. Since $|vy| > 0$, pumping $i = 2$ will increase the number of a 's and/or b 's.
 - The string s' will have the form $a^{n_a} b^{n_b} c^p d^p$, where $n_a \geq p$ or $n_b \geq p$ (and at least one is $> p$).
 - For s' to be in L , we must have $n_a = n_b = p$, which is not possible since the number of a 's and/or b 's has increased, but c 's and d 's have not.
 - Specifically, if v and y contain only a 's and b 's, then s' will have $p + k$ a 's and $p + l$ b 's, but still p c 's and p d 's, where $k + l > 0$. Since $p + k \neq p$ (or $p + l \neq p$), $s' \notin L$.
 - **Case 2: vxy is contained entirely within a single block (e.g., c^p).**
 - v and y contain only c 's. Pumping $i = 2$ yields $s' = a^p b^p c^{p+|v|+|y|} d^p$.
 - Since $|v| + |y| > 0$, the count of c 's is greater than p , but the counts of a 's, b 's, and d 's remain p .
 - Since the counts are unequal, $s' \notin L$.
5. **Conclusion:** In all possible placements of vxy , pumping s to uv^2xy^2z yields a string s' that does not have an equal number of a 's, b 's, c 's, and d 's. This contradicts the Pumping Lemma. $L = \{a^n b^n c^n d^n \mid n \geq 0\}$ is not context-free.

(14) Show that the given language is a CFL but its complement is not.

$$L = \{ a^i b^j c^k \mid i \neq j \text{ or } i \neq k \}$$

ASSIGNMENT- 5

① Find CFG for $L = \{a^i b^j c^k \mid i=j \text{ or } i=k\}$

Soln

$i=j$

$a^i b^i c^k$

↓

$S_1 \rightarrow AC$

$A \rightarrow \epsilon \mid aAb$

$C \rightarrow \epsilon \mid cC$

$i=k$

$a^i b^j c^k$

↓

$S_2 \rightarrow aS_2c \mid B$

$B \rightarrow bB \mid \epsilon$

$S \rightarrow S_1 \mid S_2$

$S_1 \rightarrow AC$

$A \rightarrow \epsilon \mid aAb$

$C \rightarrow \epsilon \mid cC$

$S_2 \rightarrow aS_2c \mid B$

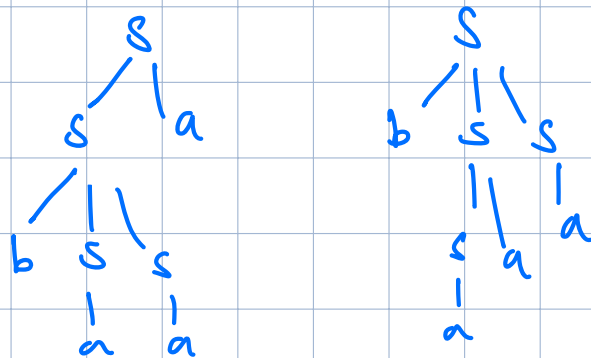
$B \rightarrow bB \mid \epsilon$

② Show that CFG with productions

$S \rightarrow a \mid Sa \mid bSs \mid sSb \mid sBs$

is ambiguous

Solⁿ consider string ba a a



since we get 2 derivative trees

CFG is ambiguous

③ Define PDA

⋮

④ Find the equivalent grammar with no null productions and unit productions in :

$S \rightarrow ABA$

$A \rightarrow aA \mid \epsilon$

$B \rightarrow bB \mid \epsilon$

Solⁿ $NV = \{A, B, S\}$

G' : $S \rightarrow ABA \mid BA \mid AB \mid AA \mid B \mid A$

$A \rightarrow aA \mid a$

$B \rightarrow bB \mid b$

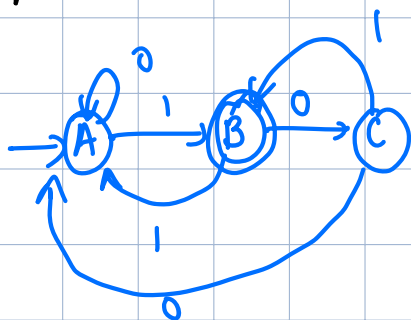
G'' : $S \rightarrow ABA \mid BA \mid AB \mid AA \mid bB \mid b \mid aA \mid a$

$A \rightarrow aA \mid a$

$B \rightarrow bB \mid b$

⑤ Construct DPDA equivalent to the DFA whose transition table is as below. Here A is the start and B is final state

	0	1
A	A	B
B	C	A
C	A	B



We construct a DPDA $M = (Q, \Sigma, \Gamma, \delta, q_0, Z_0, F)$ equivalent to the DFA:

- $Q = \{A, B, C\}$ (same states as DFA)
- $\Sigma = \{0, 1\}$ (input alphabet)
- $\Gamma = \{Z_0\}$ (stack alphabet — single bottom-of-stack symbol)
- $q_0 = A$ (start state)
- Z_0 = initial stack symbol
- $F = \{B\}$ (accepting state)

Observation: Since the DFA doesn't use a stack, the DPDA transitions **never pop**; they just read the input and move state. The stack is untouched.

Step 3 — DPDA transition function δ

- Form of δ : $\delta(q, a, y) = (p, y')$
 - q = current state
 - a = input symbol (or ϵ)
 - y = top of stack symbol
 - (p, y') = next state and stack symbol (we will just leave y unchanged)
- Let's define δ for all combinations of states, inputs, and top of stack = Z_0 :

Current State	Input	Stack top	Next state	Stack top after move $\square$
A	0	Z_0	A	Z_0
A	1	Z_0	B	Z_0
B	0	Z_0	C	Z_0
B	1	Z_0	A	Z_0
C	0	Z_0	A	Z_0
C	1	Z_0	B	Z_0

- Stack **never changes** (we just leave Z_0 on top)
- No ϵ -transitions are needed since DFA transitions are deterministic and consume input.

⑥ Define language accepted by PDA

.

:

⑦ Define CFG and the language generated by CFG. Give CFG for:

(a) language corresponds to $(abb + b)^*(ab)$

(b) $L = \{w \mid w \in \{a,b\}^* \text{ and } n_a(w) \neq n_b(w)\}$

Soln

(a) $S \rightarrow x\bar{x}$
 $X \rightarrow \epsilon \mid bx \mid abbX$
 $Y \rightarrow \epsilon \mid abY$

⑧ Convert to CNF

$$\begin{aligned} S &\rightarrow 0A \mid 1B \\ A &\rightarrow 0AA \mid 1S \mid 1 \\ B &\rightarrow 1BB \mid 0S \mid 0 \end{aligned}$$

Soln

CNF: $S \rightarrow AB$
 $S \rightarrow \alpha, \alpha \in T$

$$S \rightarrow x_0 A \mid x_1 B$$

$$x_0 \rightarrow 0$$

$$x_1 \rightarrow 1$$

$$A \rightarrow x_0 T_A \mid x_1 S \mid 1$$

$$B \rightarrow x_1 T_B \mid x_0 S \mid 0$$

$$T_A \rightarrow AA$$

$$T_B \rightarrow BB$$

(9) Convert CFG G to G' in CNF generating $L(G) \neq \emptyset$

$$A \rightarrow BC$$

$$B \rightarrow CA \mid b$$

$$C \rightarrow AB \mid a$$

Soln
=

$$B \rightarrow ABA \mid aA \mid b$$

$$C \rightarrow BCB \mid a$$

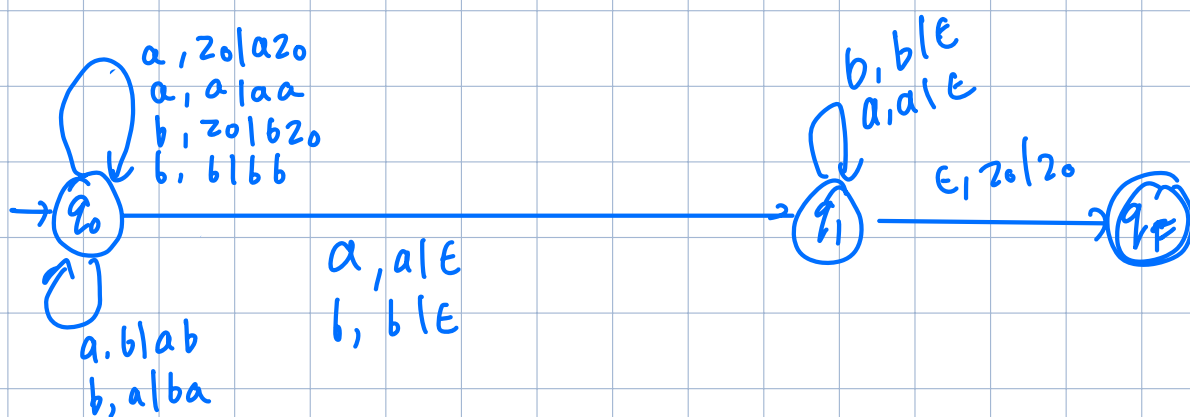
$$A \rightarrow ABA C \mid aAC \mid bC$$

$$A \rightarrow$$

⑩ Construct PDA for $L = \{ w w^R \mid w \in \{a, b\}^* \}$. Show ID that the string $abaaba$ is accepted.

Soln

$| a b a a b a \epsilon |$



$(q_0, abaaba, z_0)$



$(q_0, baaba, az_0)$



$(q_0, aaba, ba)$



(q_0, aba, ab)



(q_0, ba, aa)



$(q_1, ba, b.)$



(q_0, a, ba)

↓

(q_0, ϵ, ab)

↓
X

(q_1, a, a)

↓

(q_1, ϵ, z_0)

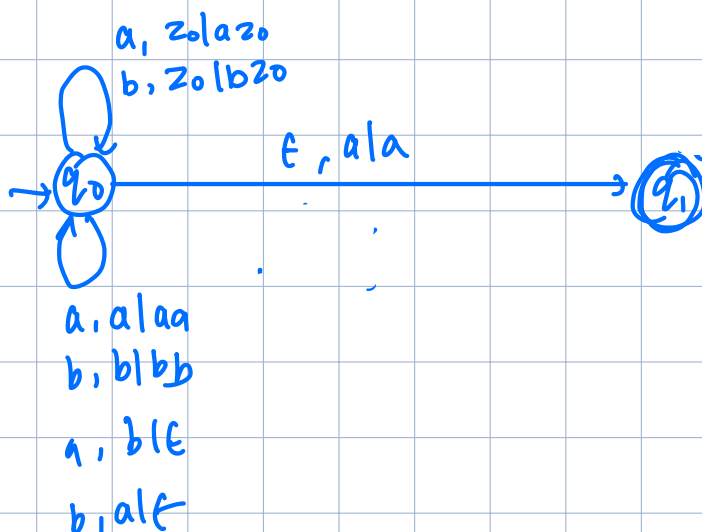
↓

(q_2, ϵ, z_0)

✓

- (ii) Construct DPDA to accept strings with more a's than b's over $\Sigma = \{a, b\}$. Show by ID that $abbb a a$ is accepted.

Soln



a b b b a a

$(q_0, abbbaaa, z_0)$

↓

$(q_0, bbbaaa, az_0)$

↓

$(q_0, bbaaa, z_0)$

↓

$(q_0, baaa, bz_0)$

↓

(q_0, aaa, bb)

↓

(q_0, aa, bz_0)

↓

(q_0, a, z_0)

↓

(q_0, ϵ, az_0)

↓

(q_1, ϵ, qz_0)

✓

- (12) What are the steps to be followed while finding an equivalent PDA for the given CFG. Find an equivalent PDA to the CFG with productions.

$$S \rightarrow aSb \mid bSa \mid aS \mid Sa \mid \epsilon$$

Soln

Given a CFG $G = (V, T, S, P)$, a standard construction of an equivalent PDA:

$$M = (Q, \Sigma, \Gamma, \delta, q_0, z_0, F)$$

$$\Sigma = T$$

$$\Gamma = V \cup \{z_0\}$$

PDA transitions:

For each production $A \rightarrow \alpha$ in a CFG:

- (1) If the top of the stack is a non-terminal A , the PDA can replace it with the RHS α (push α on stack)
- (2) If the top of the stack is a terminal matching the current input symbol, PDA pops the terminal and consumes the input.

Formally, transitions:

- for production $A \rightarrow x_1 x_2 \dots x_n$ ($x_i \in V \cup \epsilon$):

$$\delta(q, \epsilon, A) = \{(q, x_1 x_2 \dots x_n)\}$$

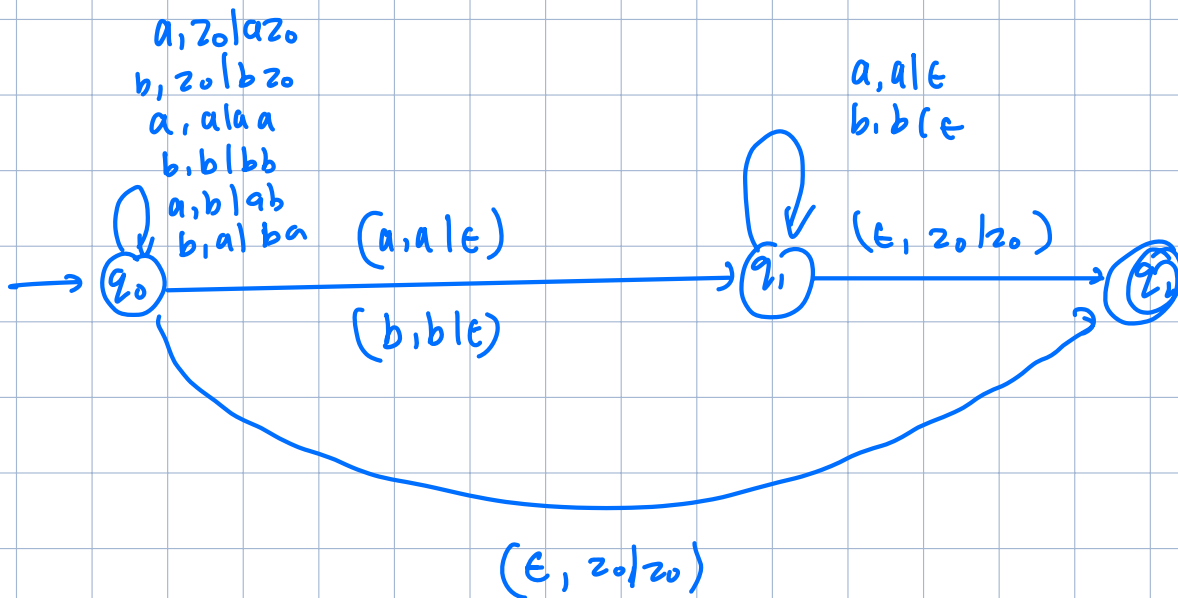
- for terminal match a :

$$\delta(q, a, a) = \{(q, \epsilon)\}$$

- start by pushing start symbol S on the stack along with z_0

$$S \rightarrow asb \mid bsa \mid abs \mid baS \mid sab \mid Sba \mid \epsilon$$

⑮ What are the steps to be follo



Pumping Lemma:

let L be any CFL

let $u \in L$, there exists an integer p such that
 $|u| \geq p$. We can decompose u as $u = vwxyz$

such that

$$|wy| \geq 1$$

$$|wxy| \leq p$$

$$\text{for } \forall i \geq 1, vw^i xy^i z \in L$$

Proof

let p be no. of variables in G

and n such that

$$n = 2^{p+1}$$

let $u \in L$

such that $|u| \geq n$

While generating derivation tree of u # of height h
 u will have at most 2^h leaves.

$$|u| \geq n \Rightarrow |u| \geq 2^{p+1}$$

$$n \geq p+2$$

$\therefore \exists$ the path from root to a leaf containing at least $p+2$ nodes

By pigeon hole principle, there will be some variables that repeat

let the repeat var = A

higher A (closer to root)

lower A (closer to leaf)

x = substring of u from lower A

wxy = substring derived from higher A.

[w and y be strings of terminal such that
substring of u from higher A
and v, z be pr ...]

If N is a node corresponding to occurrence of A

nearer root

height of subtree of both root N
 $\leq p+1$

it has $\leq 2^{p+1}$ leaves

(way) $\leq n$

Since G is in CNF ...

1
2 child

and both subtrees must derive

at least 1 term

hence A must produce 2 children
each produces term ...

$$\begin{aligned} I &\rightarrow a | b | I_a | I_b | I_0 | I_1 \\ E &\rightarrow I | E * E | E + E | (E) \end{aligned}$$

$$I \rightarrow a | b$$

$$E \rightarrow I | E * E | E + E | (E)$$

$$\delta(q, \epsilon, Z_0) = \delta(q, \epsilon)$$

$$\delta(q, \lambda, \epsilon) = (q, I)$$

$$\delta(q, \lambda, \epsilon) = (q, E * E)$$

$$\delta(q, \lambda, \epsilon) = (q, E + E)$$

$$\delta(q, \lambda, \epsilon) = (q, (E))$$

$$\delta(q, \epsilon, \epsilon) = (q, \epsilon)$$

$$\delta(q, a, a) = (q, \epsilon)$$

$$\delta(q, a, b) = (q, \epsilon)$$

$$\delta(q, +, +) =$$

~~$\delta(q, \epsilon, Z_0)$~~

$$\begin{aligned} \delta(q_0, \epsilon, Z_0) &= \delta(q, \epsilon) \\ &= (q, \epsilon) \end{aligned}$$

$$\delta(q, \lambda, 1) = (q, a)$$

$$\delta(q, \lambda, 1) = (q, b)$$

~~re~~

$$\begin{aligned} \delta(p, 0, z) &= (p, xz) \\ \delta(p, 0, x) &= (p, xx) \\ \delta(p, 1, x) &= (q, \epsilon) \\ \delta(q, 1, x) &= (q, \epsilon) \\ \delta(q, \epsilon, x) &= (q, \epsilon) \\ \delta(q, \epsilon, z) &= (q, \epsilon) \end{aligned}$$

$$\begin{aligned} s &\rightarrow [p, z_0, p] \\ &\rightarrow [p, z_0, q] \end{aligned}$$

$$\begin{aligned} [p, z, p] &\rightarrow 0 [p \ x \ p] [p \ z \ p] \\ [p, z, p] &\rightarrow 0 [p \ x \ q] [q \ z \ p] \\ [p, z, q] &\rightarrow 0 [p \ x \ q] [q \ z \ q] \\ [p, z, q] &\rightarrow 0 [p \ x \ q] [q \ z \ q] \end{aligned}$$

$$[p, z, t] \rightarrow 0 [p \ x \ r] [r \ z \ t]$$

$$\begin{aligned} r &\in Q \\ t &\in Q \end{aligned}$$

$$\gamma \begin{matrix} \nearrow p \\ \searrow q \end{matrix}$$